RTT Fairness Improvement in TCP BBR with Dynamic Target Adjustment

Yuki Kuramo
Electrical Engineering and Electronics Kogakuin University
Graduate School
Tokyo, Japan
j021117@g.kogakuin.jp

Saneyasu Yamaguchi
Department of Information and Communications Engineering
Kogakuin University
Tokyo, Japan
sane@cc.kogakuin.ac.jp

2025 IEEE International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan) | 979-8-3315-8741-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCE-Taiwan66881.2025.11208040

***Abstract*—A congestion control algorithm TCP BBR is widely used in the current Internet. It has been reported that the throughput becomes unfair when multiple TCP BBR connections with different RTTs communicate over a bottleneck link. In our previous work, we proposed a method to reduce the throughput of TCP BBR in a network element by making TCP BBRs observe small RTTs. In this paper, we propose a method to improve RTT fairness by dynamically detecting bandwidth consuming connections in a network element and evaluate the method with our prototype.**

Keywords—TCP BBR, RTT Fairness, P4, Congestion control

I. INTRODUCTION

TCP BBR [1] is one of the congestion control algorithms widely used in the current Internet. TCP BBR aims to achieve high throughput without making a queue by estimating the physical delay and the bottleneck bandwidth and using the product of these two values as the congestion window size. In the work [2], it was shown that throughput unfairness occurs when TCP BBR connections with different round-trip times (RTTs) share the same bottleneck link and communicate simultaneously. In our previous work [3], we proposed a method to reduce the throughput of a TCP BBR connection by making it observe a small RTT, and showed that applying this method to a statically determined connection could slightly improve RTT fairness. In this paper, we propose a method to improve fairness more by dynamically adjusting the target connections.

II. RELATED WORK

A. TCP BBR

TCP BBR aims for a state where the congestion window size equals the bandwidth-delay product (BDP). It estimates the physical propagation delay, which is called *RTprop*, and the bottleneck bandwidth, called *BtlBw*, during communication, and sets the congestion window size to the product of these two values, as in the following equation

$$cwnd = BtlBw \times RTprop \quad (1)$$

where *cwnd* is the congestion window size, *BtlBw* is the total acknowledged (ACK) byte size in 10 * *RTT* divided by 10 * *RTT*, and *RTprop* is the minimum *RTT* observed in 10 seconds. *RTprop* is discarded after 10 seconds since the last update. Thus, once a small *RTT* is observed, the value is held for 10 seconds and then discarded.

B. Throughput fairness in TCP

M. Hock et al. point out the RTT unfairness in TCP BBR connections [2]. Specifically, they showed that in a situation where TCP BBR connections with different RTTs share the same bottleneck link, the connection with the longer RTT consumes more bandwidth.

Packet dropping with AQM at a network element has been proposed to improve throughput fairness between TCP connections. These methods improve fairness by dropping a small number of packets at a network element and reducing the throughput of bandwidth-consuming connections. However, TCP BBR does not respond directly to packet loss, and these methods cannot control TCP BBR throughput [4].

C. QACK

We proposed the QACK method [3] to control the throughput of TCP BBRs in a network element. In this method, a network element sends an Ack packet instead of the receiver, and makes the TCP BBR observe a small RTT. We call this Ack packet a QACK (Quick ACK) packet. When TCP BBR observes QACK packets, its *RTprop* in equation (1) becomes smaller and *cwnd* also becomes smaller. Fig. 1 illustrates an overview of its behavior. In the normal case, data and ACK packets are transferred as shown in (1), (2), (3), and (4) in (a) in Fig. 1. In QACK case, as shown in (b) in Fig. 1, (1) a data packet is transmitted to the switch, (2-1) a data packet is forwarded to the receiver, and at the same time (2-2) a QACK packet is transmitted to the sender. The RTT of a QACK is smaller than that of a normal ACK, which reduces the congestion window size of the TCP BBR. The interval between QACK packets is a tuning parameter.

III. PROPOSED METHOD

A. Behavior

In this section, we propose a method to improve throughput fairness by dynamically adjusting the QACK target that reduces its throughput. The proposed method records the bandwidth consumption of each connection between a QACK packet and the next QACK packet. The method applies the next QACK to the connection with the highest bandwidth consumption during this period. This should ensure that the QACK target is always appropriate.

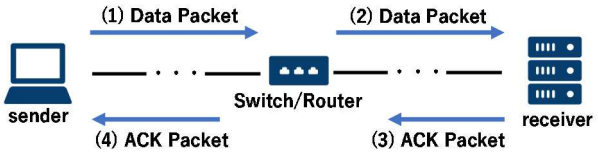
B. Implementation

We have prototyped the proposed method using P4. The prototype implementation counts the number of packets forwarded by the switch in each connection. When a QACK is applied, the QACK packet is sent to the sender of the connection with the largest counter, and each counter is reset to 0. The initial counter value at the beginning is also 0. This means that the target of the QACK changes every interval. In the prototype implementation, the QACK target is not given, i.e. it is dynamically extracted, but all passing connections that are candidates are given statically.

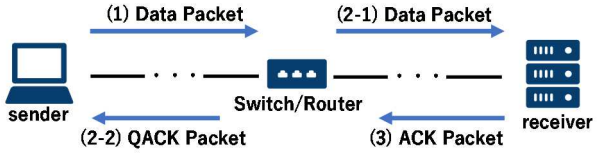
IV. EVALUATION

We applied QACK with the static method, which is an existing method, and the dynamic methods proposed in Section III, and then evaluated the fairness, in an experimental network constructed with virtual machines (VMs) provided by P4.org [5]. Fig. 2 shows the network topology, and we performed communication from Sender to Receiver1,

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on August 01,2026 at 04:48:03 UTC from IEEE Xplore. Restrictions ap



(a) normal case



(b) When sending a QACK packet

Fig. 1 QACK method behavior

Receiver2, and Receiver3 using iPerf 3.7. The communication lasted for 600 seconds, and at the middle of it, which is about 300 seconds, we enabled periodic QACK. The target of the QACK is constant with the static method, and is dynamically changed based on the proposed method with the dynamic method. To highlight the effect of QACK, the delay between a receiver and Switch is set to be significantly larger than that between Sender and Switch.

Fig. 3 depicts the average Fairness Indices of throughput without QACK, with QACK by the static method, and with QACK by the dynamic method. The QACK interval was set to 1024 packets. The results demonstrate that the application of QACK substantially improves the fairness. Comparing the static and dynamic methods, we can see that the dynamic method was even better.

Figs. 4 and 5 show the throughput trend of the static and dynamic methods with a QACK interval of 1024 packets. In Fig. 4, the target was statically set to the connection with the largest RTT based on the findings in [2], i.e., the larger the RTT, the larger the bandwidth consumption. We can see that the throughput of the second largest RTT improved its throughput significantly. We can also see that the throughput of the smallest RTT improved its throughput only slightly. On the contrary, the throughput of the largest RTT decreased remarkably after enabling QACK. In some periods, it was the lowest in the three connections. This means that the method sometimes reduces the throughput unnecessarily. On the other hand, the dynamic method avoided too much reduction.

V. Conclusion

In this paper, we proposed an improvement to an existing method to reduce the throughput of TCP BBR, which is called QACK. The proposed method dynamically changes the target of the QACK for throughput reduction. We then implemented a prototype of the proposed method and evaluated the RTT fairness between TCP BBR connections in a VM environment. The results show that the proposed method can improve the RTT fairness of TCP BBRs compared to the existing method.

For future work, we plan to implement the proposed method in a physical environment, evaluate its performance in different setups, and evaluate its scalability and overhead in the switch.

References

[1] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, Van Jacobson, “Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time,” ACM Queue, vol. 14, Issue 5, pp. 20-53, 2016.

[2] M. Hock, R. Bless and M. Zitterbart, "Experimental evaluation of BBR congestion control," 2017 IEEE 25th International Conference on Network Protocols (ICNP), Toronto, ON, Canada, 2017, pp. 1-10, doi: 10.1109/ICNP.2017.8117540.

[3] Yuki Kuramo, Saneyasu Yamaguchi, “Control TCP BBR Throughput in a Programmable Switch” (*submitted* for IEEE HPSR CTN-SRMI 2025)

[4] K. Sasaki, M. Hanai, K. Miyazawa, A. Kobayashi, N. Oda and S. Yamaguchi, "TCP Fairness Among Modern TCP Congestion Control Algorithms Including TCP BBR," 2018 IEEE 7th International Conference on Cloud Networking (CloudNet), Tokyo, Japan, 2018, pp. 1-4, doi: 10.1109/CloudNet.2018.8549505.

[5] github.com/p4lang, "P4 Tutrial", https://github.com/p4lang/tutorials, Accessed on March. 15, 2025

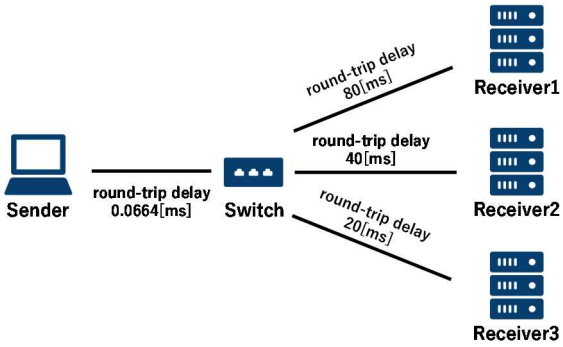


Fig. 2 Experimental network topology

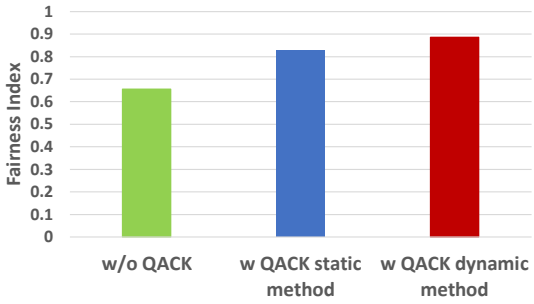


Fig. 3 Fairness Index

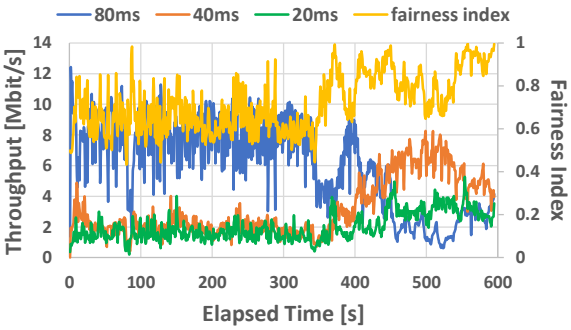


Fig. 4 Throughput trend (static method)

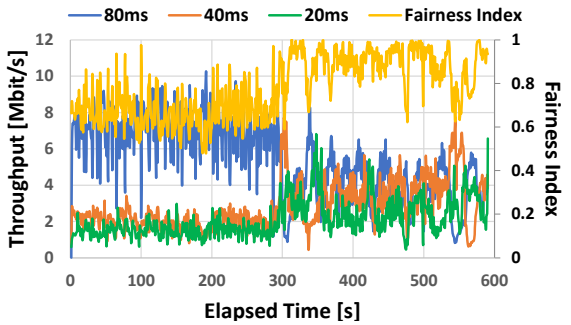


Fig. 5 Throughput trend (dynamic method)

horized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on August 01,2026 at 04:48:03 UTC from IEEE Xplore. Restrictions ap